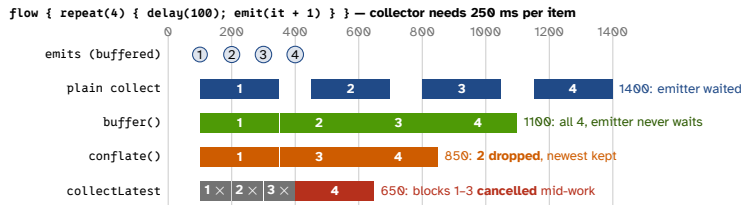
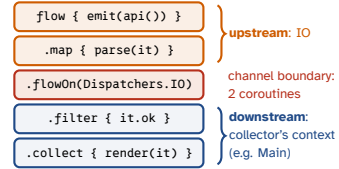


**In one line:** A Flow is a **cold, sequential** stream: collect runs the producer *in the collector's coroutine*, each value passes every operator before the next is produced, and the producer suspends while the collector works (natural backpressure). Two rules make operators composable: **context preservation** (only flowOn changes where upstream runs) and **exception transparency** (never emit from inside a catch). Concurrency and hotness are *opt-in*: buffer, flowOn, channels, stateIn/shareIn.



buffer(n) = producer and collector in **two coroutines** joined by a channel (default 64); conflate() = buffer(CONFLATED); collectLatest / mapLatest / flatMapLatest cancel the running block on each new value.

Where each line runs



A 2nd flowOn lower down only covers the lines between the two. flowOn+buffer fuse into **one** channel.

## The contract

- Intermediate** ops (map, filter, transform, onEach, debounce) return a new Flow, run nothing. **Terminal** ops (collect, first, single, toList, reduce) are suspend and start it; launchIn(scope) = scope.launch { collect() }.
- Context preservation:** emit only from the collecting coroutine — withContext/launch inside flow {} throws IllegalStateException ("Flow invariant is violated"). Other threads/ coroutines → channelFlow/callbackFlow.
- Exception transparency:** if emit throws (downstream failed), stop and rethrow; never swallow it with try/catch around emit. Use catch {} — it sees **upstream** only and may emit a fallback.
- Don't implement Flow yourself (not stable for inheritance) — use flow {} or AbstractFlow.

## Combining

|               |                                                                                                                 |
|---------------|-----------------------------------------------------------------------------------------------------------------|
| combine(a,b)  | latest of each; first output once <i>all</i> have emitted; then on any emission — UI state from several sources |
| zip(a,b)      | pairs by index (1st with 1st); ends when either ends                                                            |
| merge(a,b)    | interleaves values as they come, no pairing                                                                     |
| flatMapConcat | inner flows one after another (order kept)                                                                      |
| flatMapMerge  | inner flows concurrently, concurrency = 16 by default                                                           |
| flatMapLatest | new outer value <b>cancels</b> the running inner flow — search box                                              |

## Errors & completion

- retry(3) { it is IOException } / retryWhen { cause, attempt -> ... } *re-collect upstream* (cold ⇒ re-runs the request).
- onCompletion { cause -> }: runs on success, failure or cancel, sees upstream *and* downstream errors, does *not* handle them.
- Collector exceptions: move the work into onEach, then catch, then launchIn.

## Example — callback, search, screen state

```
fun LocationClient.updates(): Flow<Loc> = callbackFlow {
    val cb = LocationCallback { loc -> trySend(loc) } // non-suspending
    requestUpdates(cb)
    awaitClose { removeUpdates(cb) } // REQUIRED: else ISE on close
}.conflate() // keep only the newest fix
val results = query // MutableStateFlow<String>
    .debounce(300).distinctUntilChanged()
    .flatMapLatest { q -> repo.search(q) } // old search cancelled
    .catch { emit(emptyList()) }
val ui = combine(results, prefs) { r, p -> Ui(r, p) }
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), Ui())
```

## Hot flows — the knobs

| StateFlow        | SharedFlow (replay, extraBufferCapacity, onBufferOverflow = 0, 0, SUSPEND) |
|------------------|----------------------------------------------------------------------------|
| read equal value | .value; update{} = CAS                                                     |
| slow sub         | <b>not</b> emitted<br>jumps to newest                                      |
| no subs          | value kept                                                                 |

StateFlow = SharedFlow(replay 1, DROP\_OLDEST) + distinctUntilChanged + initial value; neither ever completes.

- stateIn/shareIn: one cold upstream → one shared collection. Eagerly (now, forever), Lazily (1st subscriber, forever), WhileSubscribed(stopTimeoutMillis=0, replayExpirationMillis=MAX). **Why 5 000:** a rotation gap is far shorter, so upstream survives it; in background it stops after 5 s.
- Channels** (hot, each element to *one* receiver): Channel() = RENDEZVOUS (0) · BUFFERED (64) · CONFLATED · UNLIMITED. callbackFlow/channelFlow buffer 64; channelFlow may launch concurrent senders.

## Testing

- runTest: virtual time (delay/debounce skipped; 60 s timeout); advanceTimeBy/advanceUntilIdle; endless collectors in backgroundScope; Dispatchers.setMain(testDispatcher).
- Turbine:** flow.test { awaitItem(); awaitComplete() }, expectNoEvents(). Assert a StateFlow's *values* — conflation may skip some.

## Traps

- MutableSharedFlow<Event>() + tryEmit: with a subscriber it **fails** (no buffer) — set extraBufferCapacity, or emit.
- state.value = list.apply { add(x) } — same instance, equals true: nothing emitted. Publish a **new** value.
- fun items() = flow.stateIn( ... ) — a new shared coroutine per call; use a val.
- catch never sees the collector's own exceptions.

**Remember:** cold + sequential · flowOn = upstream · buffer splits, conflate drops, latest cancels.